

CF Features Implementation — Frontend & Backend

Project: Smart AgriTech / EmbedAloT EMS

Scope: Custom dashboards, access groups, device groups, facilities hierarchy, power-flow mind map, live telemetry wiring, and related ACL hardening

Commit (main): `8340baf` — *Add CF dashboards, access/device groups, and live EMS telemetry wiring*

This document describes everything that was **added or changed** on the **web frontend** (`web_frontend/`) and **EMS backend** (`ems/ems-backend/`). Related Flutter dummy-data work is summarized briefly at the end.

Table of contents

- 1. [Overview](#)
- 2. [Backend — database & schema](#)
- 3. [Backend — APIs & controllers](#)
- 4. [Backend — authorization & live data](#)
- 5. [Frontend — API client & navigation](#)
- 6. [Frontend — features & pages](#)
- 7. [Frontend — live widgets & utilities](#)
- 8. [Role matrix](#)
- 9. [How to apply database changes](#)
- 10. [How to verify](#)
- 11. [Related Flutter work](#)
- 12. [File index](#)

1. Overview

We ported and productionized CF-dashboard capabilities into the existing EMS stack:

Feature	Backend	Frontend
Access Groups	CRUD + device/user membership	Admin + Org pages
Device Groups	CRUD + device membership	Admin + Org pages

Feature	Backend	Frontend
Facilities hierarchy	Tree CRUD, replace, device linking	Org Settings builder + device linker
Custom Dashboard Builder	Persist layout/widgets/context	Drag-drop editor for Admin / Org / User
Power Flow Mind Map	Live loads + derived sources	Org Dashboard visualization
Live telemetry	Existing sensor APIs + ACL	Widgets use live aggregates / latest / alarms
Access control	USER = DeviceUser U AccessGroup	Device lists / sensor / AI / anomalies respect ACL

Design goals

- Reuse existing sensor, anomaly, and slab-rate APIs (no parallel fake telemetry stack in production paths).
- Keep Timescale-compatible DB changes via SQL file when `prisma db push` is blocked.
- Role-aware UI: Super Admin, Org Admin, User.

2. Backend — database & schema

2.1 New Prisma models

Defined in `ems/ems-backend/prisma/schema.prisma` and mirrored in `prisma/add_cf_features.sql`.

Access Groups

Model	Table	Purpose
<code>AccessGroup</code>	<code>access_groups</code>	Named group of devices + users in an org
<code>AccessGroupDevice</code>	<code>access_group_devices</code>	Join: group ↔ device
<code>AccessGroupUser</code>	<code>access_group_users</code>	Join: group ↔ user

Device Groups

Model	Table	Purpose
<code>DeviceGroup</code>	<code>device_groups</code>	Named category for power-flow / org views
<code>DeviceGroupDevice</code>	<code>device_group_devices</code>	Join: group ↔ device
<code>DeviceGroupUser</code>	<code>device_group_users</code>	Optional user membership (schema ready)

Facilities

Model	Table	Purpose
FacilityNode	facility_nodes	Org → campus → building → floor → ... tree
FacilityNodeDevice	facility_node_devices	Link EMS devices to a facility node

FacilityNodeType enum: ORGANIZATION , CAMPUS , SITE , BUILDING , BLOCK , WING , FLOOR , DEPARTMENT , SECTION , ROOM .

Custom dashboards & power flow

Model	Table	Purpose
CustomDashboard	custom_dashboards	Name, visibility, JSON context / layout / widgets , optional targetDeviceId
PowerFlowConfig	power_flow_configs	Per-org JSON sources + savings

DashboardVisibility : PRIVATE | SHARED .

2.2 Device model relations added

On Device :

- accessGroupDevices
- deviceGroupDevices
- facilityNodeDevices

2.3 Time-range extension

In `ems/ems-backend/utils/helpers.js` :

Key	Duration	Chart bucket
1h	1 hour	1 min
24h	1 day	1 hour
7d	7 days	1 day
30d	30 days	1 day
365d (new)	~365 days	1 week

Used by sensor aggregate / history windows so custom dashboard **year** widgets map to real yearly data.

3. Backend — APIs & controllers

Routes mounted in `ems/ems-backend/routes/index.js` :

```
/access-groups
/device-groups
/facilities
/custom-dashboards  (+ /custom-dashboards/power-flow)
```

3.1 Access Groups — `/access-groups`

Files: `routes/accessGroups.js` , `controllers/accessGroupController.js`

Roles: `SUPER_ADMIN` | `ORG_ADMIN`

Method	Path	Behavior
GET	/	List groups (org-scoped), with <code>deviceIds</code> , <code>userIds</code> , nested devices/users
POST	/	Create group + memberships
PUT	/:id	Update name; replace-all <code>deviceIds</code> / <code>userIds</code>
DELETE	/:id	Delete group (cascades joins)

3.2 Device Groups — `/device-groups`

Files: `routes/deviceGroups.js` , `controllers/deviceGroupController.js`

Roles: `SUPER_ADMIN` | `ORG_ADMIN`

Method	Path	Behavior
GET	/	List active groups + devices
POST	/	Create group + device memberships
PUT	/:id	Update metadata / devices / active flag
DELETE	/:id	Delete group

3.3 Facilities — `/facilities`

Files: `routes/facilities.js` , `controllers/facilityController.js`

Method	Path	Roles	Behavior
GET	/	Auth	Tree + flat nodes; each node includes <code>deviceIds</code> / <code>devices</code>

Method	Path	Roles	Behavior
POST	/	Admin	Create node (optional <code>deviceIds</code>)
PUT	/replace	Admin	Replace entire org tree; preserves <code>deviceIds</code> from payload
PUT	/:id/devices	Admin	Set devices for one node
PUT	/:id	Admin	Update node fields + optional <code>deviceIds</code>
DELETE	/:id	Admin	Delete node (cascades children / links)

3.4 Custom dashboards — `/custom-dashboards`

Files: `routes/customDashboards.js` , `controllers/customDashboardController.js`

Method	Path	Roles	Behavior
GET	/	Auth	List; USER sees own + <code>SHARED</code>
GET	/:id	Auth	Get one dashboard
POST	/	Auth	Create (owner = current user)
PUT	/:id	Auth	Update layout/widgets/context/visibility
DELETE	/:id	Auth	Delete
GET	/power-flow	Auth	Live power-flow payload (see below)
PUT	/power-flow	Admin	Save sources / savings config

3.5 Power-flow response (live)

GET `/custom-dashboards/power-flow` now returns:

- `groups[]` — device groups with `deviceIds` , `devices` , `loadKw` / `load` (sum of live `ActivePower` / fallback `PowerConsumption` from Redis `device:{id}:latest` , then DB `currentValue`)
- `sources[]` — grid / solar / generator:
 - **Solar** from org `ExportPower` / `SolarPower`
 - **Grid** $\approx$ total load – solar – manual generator
 - Generator keeps manual edits unless live-derived
- `totalLoadKw` , `solarKw` , `gridKw`
- `savings` from stored config

4. Backend — authorization & live data

4.1 Shared device ACL — `utils/deviceAccess.js`

USER may access a device if **either**:

1. Direct assignment via `DeviceUser` , **or**
2. Membership in an `AccessGroup` that includes that device

Exports:

- `userDeviceAccessWhere(userId)`
- `deviceWhereForUser(user, extra)`
- `listAccessibleDeviceIds(user)`
- `assertDeviceAccess(deviceId, user)`

4.2 Controllers updated to use ACL

Controller	Change
<code>deviceController.js</code>	<code>getDevices</code> / <code>getDevice</code> use <code>DeviceUser</code> U <code>AccessGroup</code>
<code>sensorDataController.js</code>	<code>authoriseDevice</code> → <code>assertDeviceAccess</code>
<code>aiAnalyticsController.js</code>	Voltage / current / PF / energy / predictions assert device access
<code>anomalyController.js</code>	Single-device asserts ACL; list for USER filtered to accessible device IDs

4.3 Why this matters

Previously a USER in an org could often read **any org device** via sensor/AI endpoints if they knew the `deviceId` . Access groups are now enforceable end-to-end for listing and telemetry.

5. Frontend — API client & navigation

5.1 `web_frontend/src/api/emsApi.js` additions

Method	Endpoint
<code>getAccessGroups</code> / <code>createAccessGroup</code> / <code>updateAccessGroup</code> / <code>deleteAccessGroup</code>	<code>/access-groups</code>
<code>getDeviceGroups</code> / <code>createDeviceGroup</code> / <code>updateDeviceGroup</code> / <code>deleteDeviceGroup</code>	<code>/device-groups</code>

Method	Endpoint
<code>getFacilityTree</code> / <code>createFacilityNode</code> / <code>updateFacilityNode</code> / <code>deleteFacilityNode</code>	<code>/facilities</code>
<code>replaceFacilityTree</code>	<code>PUT /facilities/replace</code>
<code>setFacilityDevices</code>	<code>PUT /facilities/:id/devices</code>
<code>getCustomDashboards</code> / <code>getCustomDashboard</code> / <code>createCustomDashboard</code> / <code>updateCustomDashboard</code> / <code>deleteCustomDashboard</code>	<code>/custom-dashboards</code>
<code>getPowerFlow</code> / <code>updatePowerFlow</code>	<code>/custom-dashboards/power-flow</code>

5.2 Dependency

- `react-grid-layout` added in `package.json` (grid canvas for dashboard builder)
- CSS imported in `main.jsx`

5.3 Navigation (`config/navConfig.jsx`)

Role	New nav items
Admin	Custom Dashboards, Access Groups, Device Groups
Org	Custom Dashboards, Access Groups, Device Groups
User	Custom Dashboards

5.4 Routes (`App.jsx`)

Role prefix	Paths
<code>/admin/...</code>	<code>access-groups</code> , <code>device-groups</code> , <code>custom-dashboard</code> , <code>custom-dashboard/:id</code>
<code>/org/...</code>	same + power flow lives on Org Dashboard
<code>/user/...</code>	<code>custom-dashboard</code> , <code>custom-dashboard/:id</code>

6. Frontend — features & pages

6.1 Access Groups

File	Role
<code>pages/shared/AccessGroupsPage.jsx</code>	Shared CRUD UI (name, devices, users)
<code>pages/admin/AdminAccessGroups.jsx</code>	Admin wrapper
<code>pages/org/OrgAccessGroups.jsx</code>	Org wrapper

Behavior: Create/edit groups; assign devices and users; delete. Backend enforces membership for USER device visibility.

6.2 Device Groups

File	Role
<code>pages/shared/DeviceGroupsPage.jsx</code>	Shared CRUD UI
<code>pages/admin/AdminDeviceGroups.jsx</code>	Admin wrapper
<code>pages/org/OrgDeviceGroups.jsx</code>	Org wrapper

Behavior: Name/description/devices; used by Power Flow mind map as load branches.

6.3 Facilities (Org Settings)

File	Role
<code>components/facility/HierarchyEditor.jsx</code>	Quick builder (comma-separated levels)
<code>components/facility/FacilityDeviceLinker.jsx</code>	Checkbox link of devices ↔ facility nodes
<code>pages/org/OrgSettings.jsx</code>	Saves tree via <code>replaceFacilityTree</code> (includes <code>deviceIds</code>)
<code>data/facilitiesHierarchy.js</code>	API↔UI type maps, tree flatten/map, scope device helpers

Behavior:

1. Define hierarchy (campus / building / floor / department / ...).
2. Link devices to nodes.
3. Save — device links are preserved across name-based rebuilds and sent on replace.

Helpers added:

- `collectDeviceIdsFromNode`
- `resolveScopeDeviceIds`
- `mapTreeFromApi` / `flattenTreeForApi` now carry `deviceIds`

6.4 Custom Dashboard Builder

File	Role
<code>pages/shared/DashboardList.jsx</code>	List / create / open dashboards
<code>pages/shared/DashboardEditor.jsx</code>	Full editor (layout, widgets, context, save)
<code>components/dashboard-builder/*</code>	Canvas, toolbar, widgets, settings, filters
<code>data/widgetCatalog.js</code>	Widget types, metrics, color themes
<code>utils/customDashboardHelpers.js</code>	Templates / defaults helpers

Widget types: line, area, bar, pie, gauge, stat, table, alarms, heatmap, text/markdown, multiseries.

Metrics: energy consumption, active power, voltage, current, power factor, cost, carbon, devices online, active alarms.

Context filters: building / floor / department / time range / target device (inherited or per-widget override).

6.5 Power Flow Mind Map

File	Role
<code>components/ui/PowerFlowMindMap.jsx</code>	Sources → total load → device group cards
<code>pages/org/OrgDashboard.jsx</code>	Loads <code>getPowerFlow</code> , shows live <code>load</code> / <code>loadKw</code> , editable sources

7. Frontend — live widgets & utilities

7.1 `utils/widgetLiveData.js`

Replaces seeded mock series for production rendering:

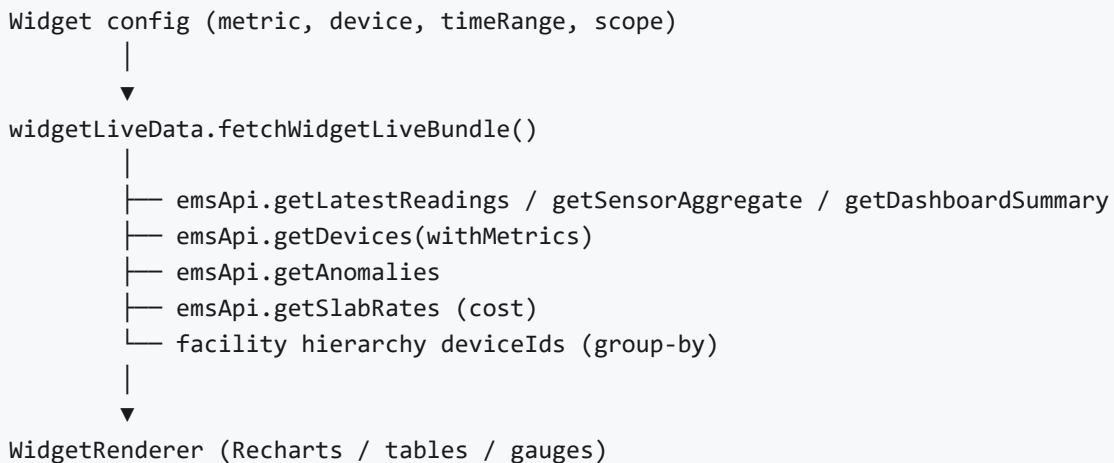
Concern	Implementation
Time map	<code>today→24h</code> , <code>week→7d</code> , <code>month→30d</code> , <code>year→365d</code>
Metric → variable	e.g. <code>activePower→ActivePower</code> , <code>energyConsumption→PowerConsumption</code>

Concern	Implementation
Cost	kWh × average org slab rate (<code>getSlabRates</code>), fallback PKR 45
Carbon	kWh × 0.45 kg CO ₂
Series	<code>getSensorAggregate</code> (+ summary chart fallback)
Current	<code>getLatestReadings</code>
Table / comparison	<code>getDevices({ withMetrics: true })</code> ; facility group-by sums linked devices
Alarms	<code>getAnomalies</code> (active)
Heatmap	7d aggregate → day×hour matrix

7.2 `components/dashboard-builder/WidgetRenderer.jsx`

- Async live bundle load
- Loading / empty states
- **Refresh:** poll every **30s**
- **Socket:** `reading:new` / `alarm:new` when socket enabled and device subscribed

7.3 Data flow (widget)



8. Role matrix

Capability	Super Admin	Org Admin	User
Access Groups CRUD	✓	✓	✗
Device Groups CRUD	✓	✓	✗
Facility tree + device link	✓*	✓	✗

Capability	Super Admin	Org Admin	User
Custom dashboards	✓	✓	✓ (own + shared)
Power Flow view	—	✓ (Org Dashboard)	—
Power Flow edit sources	✓	✓	✗
Devices via Access Group	—	—	✓ (enforced)
Live widget telemetry	✓	✓	✓ (ACL)

* Super Admin typically scopes by `organizationId` query / body where required.

9. How to apply database changes

Normal `prisma db push` may fail on existing Timescale hypertables. Use the SQL companion file:

```
cd ems/ems-backend
npx prisma generate
npx prisma db execute --file prisma/add_cf_features.sql
```

This creates CF tables **including** `facility_node_devices` (idempotent `IF NOT EXISTS`).

Restart the backend after generate + SQL.

10. How to verify

1. **Backend** starts without module errors; Prisma client includes new models.
2. **Org Admin**
 - Create Access Group (users + devices) → login as USER → only those devices appear.
 - Create Device Groups → Org Dashboard Power Flow shows non-zero loads when telemetry exists.
 - Org Settings → hierarchy + link devices → Save.
 - Custom Dashboard → add widgets → select device → charts show live series.

3. **Frontend build**

```
cd web_frontend && npm run build
```

4. Confirm widgets refresh (wait ~30s or push a reading over MQTT/ingest).

11. Related Flutter work

Not part of the web CF UI, but included in the same delivery commit:

Item	Location	Notes
Dummy data mode	<code>app/lib/config/app_config.dart</code> , <code>app/lib/data/dummy/*</code>	<code>USE_DUMMY_DATA</code> / force flag; intercepts API
Auth / API hooks	<code>auth_service</code> , <code>api_client</code> , <code>ems_api</code> , <code>socket_service</code>	Dummy-aware paths
Login	<code>login_page.dart</code>	Role demo credentials support

Dummy mode is **Flutter-only** (web still talks to live backend).

12. File index

Backend (new)

```
ems/ems-backend/controllers/accessGroupController.js
ems/ems-backend/controllers/deviceGroupController.js
ems/ems-backend/controllers/facilityController.js
ems/ems-backend/controllers/customDashboardController.js
ems/ems-backend/routes/accessGroups.js
ems/ems-backend/routes/deviceGroups.js
ems/ems-backend/routes/facilities.js
ems/ems-backend/routes/customDashboards.js
ems/ems-backend/utils/deviceAccess.js
ems/ems-backend/prisma/add_cf_features.sql
```

Backend (modified)

```
ems/ems-backend/prisma/schema.prisma
ems/ems-backend/routes/index.js
ems/ems-backend/utils/helpers.js
ems/ems-backend/controllers/deviceController.js
ems/ems-backend/controllers/sensorDataController.js
ems/ems-backend/controllers/aiAnalyticsController.js
ems/ems-backend/controllers/anomalyController.js
```

Frontend (new)

```
web_frontend/src/pages/shared/AccessGroupsPage.jsx
web_frontend/src/pages/shared/DeviceGroupsPage.jsx
web_frontend/src/pages/shared/DashboardList.jsx
web_frontend/src/pages/shared/DashboardEditor.jsx
web_frontend/src/pages/admin/AdminAccessGroups.jsx
web_frontend/src/pages/admin/AdminDeviceGroups.jsx
web_frontend/src/pages/org/OrgAccessGroups.jsx
web_frontend/src/pages/org/OrgDeviceGroups.jsx
web_frontend/src/components/dashboard-builder/* (AddWidget, ContextFilter, GridCanvas, ...)
web_frontend/src/components/facility/HierarchyEditor.jsx
web_frontend/src/components/facility/FacilityDeviceLinker.jsx
web_frontend/src/components/ui/PowerFlowMindMap.jsx
web_frontend/src/data/facilitiesHierarchy.js
web_frontend/src/data/widgetCatalog.js
web_frontend/src/utils/widgetLiveData.js
web_frontend/src/utils/customDashboardHelpers.js
```

Frontend (modified)

```
web_frontend/package.json
web_frontend/package-lock.json
web_frontend/src/main.jsx
web_frontend/src/App.jsx
web_frontend/src/api/emsApi.js
web_frontend/src/config/navConfig.jsx
web_frontend/src/pages/org/OrgDashboard.jsx
web_frontend/src/pages/org/OrgSettings.jsx
```

Summary

Backend: New CF schema + REST APIs; access-group ACL on devices/sensors/AI/anomalies; live power-flow aggregation; facility↔device links; `365d` aggregates.

Frontend: Full CF UI for groups, facilities, custom dashboards, and power flow; widgets driven by live EMS APIs with polling + sockets; cost from slab rates; facility group-by using linked devices.

For older platform docs (architecture, deployment, full ERD), see the numbered guides in this `docs/` folder.